Training Time? Search versus Self-Play PPO Agent in Checkers

Clark Kitchen
CS 681

October 25, 2026

1 Introduction

The game play of two player games has drawn attention from academics as a popular testing ground for artificial intelligence since Shannon (1950). Some two player games have more than millions of strategic moves to master like Go (Tromp and Farnebäck, 2016), while others like checkers have been mastered by using alpha-beta search (Schaeffer et al., 1996). With the rise of neural networks like AlphaZero, researchers have focused efforts on using neural networks to play two player games (Silver et al., 2017). This research proposal compares search algorithms to neural networks in playing the two player game checkers. The goal is to look at the cost of training self-playing networks compared to already established search algorithms and compare when self training is worth the cost.

2 Problem Definition and Background

The game of checkers can be described as a zero sum two player game where there is no fog of war both players can see the entire board. The game is also deterministic; there is no dice or chance involved in the game. This project utilizes the World Checkers Draughts Federation's Rules of Draughts (Checkers) (WCDF, n.d.) . This rule set covers using an 8×8 board, rules regarding multi jump moves within a player's single move, and how a king is made and can move around the board. Another rule is that captures are mandatory, this cuts down on the number of legal moves as moves when captures are available are forced to capture. These rules are the same rules used in Schaeffer et al. (2007). These characteristics are what make search work so well. The ability to see the entire board and the game being zero sum allows for deterministic search algorithms the ability to solve the game.

The game of checkers can be represented as a game tree. The positions are nodes and legal moves are edges. In the tree b is the average number of legal moves per position. Then d is the number of moves ahead the search looks, with a multi-jump then counting as just one move (Rich, 2008, §45.2.3).

$$V(s) = \begin{cases} u(s) & \text{if } s \text{ is terminal} \\ \max_{a \in A(s)} V(T(s,a)) & \text{if MAX is to move} \\ \min_{a \in A(s)} V(T(s,a)) & \text{if MIN is to move} \end{cases} \tag{1}$$

When looking at the equation $V(s)$ represents the minimax value of s. This is the value if both players play perfectly from that point. Next, $u(s)$ is the outcome of a finished game, scored from MAX's point of view. A win is $+1$, 0 for a draw and -1 for a loss. $A(s)$ is the set of legal moves in position s. Under the rules this includes the mandatory forced capture moves referenced above.

$T(s, a)$ is the position that results from playing move a in position s. In this equation T stands for transition. When search is used the search will stop at depth d and then score positions with an evaluation function. The minimax agent used the material count men + 2 * kings. Minimax examines about $O(b^d)$ positions, and alpha-beta pruning in the best case about $O(b^{d/2})$ Knuth and Moore (1975).

For checkers, the problem of checking if the player to move can force a win from a given position on an $N \times N$ board is EXPTIME-complete (Robson, 1984). EXPTIME is a class of problems a computer can solve, but the time it takes to solve can grow exponentially with input size. EXPTIME-complete problems are the hardest problems in this class which includes checkers. This means it is provably intractable as the board size grows (Rich, 2008, §28.9), but this does not apply to the 8×8 board. When the 8×8 board is used with about 5×10^{20} positions it was solved as a draw (Schaeffer et al., 2007).

Given that a checkers program cannot search to the end for every move, it must look only a few moves ahead and use an evaluation function to judge positions where it stops. Is it better to train a neural network once, or search deeper every move? This research looks to determine which costs less.

3 History, Current Status, and Our PPO Agent

The research into building checkers programs happened in 1959 with Samuel building the first checkers program that played itself and improved from its gameplay (Samuel, 1959). Chinook in 1996 used the alpha-beta search, tuned evaluation functions and endgame databases to become the World Man-Machine Champion in 1994 (Schaeffer et al., 1996). In 2007 Schaeffer wrote deep search implicitly uncovers knowledge when solving by search plus databases covering every position with 10 or fewer pieces 3.9×10^{13} positions in all (Schaeffer et al., 2007). In 2017 AlphaZero learned from self-play using the rules and searched 80 thousand positions per second against Stockfish's 70 million, due to a selective network (Silver et al., 2017). Jones in 2021 used Hex to measure the trade off directly, calculating train time against test time compute and how certain amounts of each can be eliminated against the other (Jones, 2021). This project will aim to measure the training compute vs. search compute exchange rate for checkers.

Currently a neural network was developed using a method that limits how far the policy can change in each update by clipping (Schulman et al., 2017). The agent developed learned only from self-play, with no expert games or a search function. The network outputs a policy over 128 actions and a value score in reference with Table 1. Illegal moves have been masked out. This is directly $A(s)$ from section two's equation.

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\Big)\Big], \qquad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{2}$$

In equation 2 it shows that $r_t(\theta)$, the probability ratio, $\hat{A}_t$, is the advantage of how much better the move turned out to be while $\epsilon = 0.2$, is the clip range, and $\mathbb{E}_t$ the average over all moves in the training batch. In Table 2 you can see the results of the PPO Agent training. The final checkpoint scored worse than update 4,608 (0.8611 vs. 0.9005). This value head is a learned evaluation function so it replaces material count inside the minimax of Eq 1. This sets up a hybrid agent.

Table 1: PPO agent: network and training setup.

Parameters	470,410
Input	$8 \times 8 \times 8$ board planes (mover's perspective)
Network	64-channel 3×3 conv stem + 6 residual blocks
Outputs	128 masked policy logits; value $\in [-1, 1]$
Training	Self-play PPO with a league of past policies
Updates	6,144 $\times$ 8,192 transitions (50.3M total)
Selected checkpoint	Update 4,608 (37.7M transitions)
Hyperparameters	$\gamma = 1.0$, $\lambda = 0.95$, $\epsilon = 0.2$, lr 3×10^{-4}
Hardware / time	1$\times$ RTX 5070 (12 GB), 21.6 h

Table 2: Preliminary results: 216 fixed openings $\times$ 2 colors (432 games per row).

Checkpoint	Opponent	W / D / L	Score	95% interval
4,608	Random	432 / 0 / 0	1.000	0.991–1.000
4,608	Minimax-2	354 / 70 / 8	0.9005	0.869–0.925
6,144	Minimax-2	327 / 90 / 15	0.8611	0.825–0.891

4 Research Question and Hypothesis

This project's goal is to measure the exchange rate between training compute and search compute. This means that there is an exchange rate that operates between the compute used for training and the compute used for search. Jones (2021) found in Hex, that ten times the training can replace about fifteen times of search. This project's hypothesis is that more training can reduce the search needed for the same playing strength. This hypothesis is rejected if more training does not reduce the search need for equal strength or there is no consistent relationship across training levels.

5 Goals and Methodology

To test the hypothesis the research project will complete a nine checkpoint long training run measured evenly on a log scale: updates 24, 48, 96, 192, 384, 768, 1,536, 3,072, and 6,144 (0.2M to 50.3M self-play transitions). The same configuration as Table 1 will be used with a different seed. At the depth of 0 the agent uses the PPO policy alone with no search. The deeper levels alpha-beta uses the checkpoints value head to score the positions at the end of the search. The training measurement will be in self-play transitions and in search positions evaluated per move. Alpha-beta search will only count pieces that will serve as a reference point without any training at all.

Every agent on the grid will face the same opponent. A test run will be used to pick an opponent that's not too easy or hard. Games will start from fixed openings and each opening will be played twice once as each color. An agent's score is its wins plus half its draws, divided by the number of games.

The exchange rate is found by finding target score S^*. For each of the nine checkpoints, the smallest amount of search that reaches S^* will be recorded, and these points are fitted with EQ 3.

$$\log_{10} n_{S^*}(C) = \alpha - \beta \log_{10} C \tag{3}$$

In this equation C represents the training compute in self-play transitions, and α represents the intercept of the fitted line. Looking at the slope of that line β, this is the exchange rate which shows every time training increases tenfold. To check if the number is reliable the calculation will be repeated 1,000 times on randomly resampled openings. The main limitation is that all nine of the checkpoints will come from a single training run.

6 Conclusion

This project will measure how much search training can replace in checkers using a self-play PPO agent as the trained part. The result will be a single number, the exchange rate. This turns the question of " is training a network worth it?" into something that can be measured. Either outcome will be useful as it will answer a question that will improve the optimization of checkers play and could be applied to similar games or problems. The main limitations of this project are a single self-play training run, only applied to 8×8 checkers and this agent design. Future work could add more training runs and check agents moves using an endgame database.

References

Andy L. Jones. Scaling scaling laws with board games. arXiv preprint arXiv:2104.03113, 2021. URL `https://arxiv.org/abs/2104.03113`.

Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4): 293–326, 1975. doi: 10.1016/0004-3702(75)90019-3.

Elaine Rich. *Automata, Computability and Complexity: Theory and Applications*. Pearson Prentice Hall, Upper Saddle River, NJ, 2008.

J. M. Robson. N by N checkers is Exptime complete. *SIAM Journal on Computing*, 13(2):252–267, 1984. doi: 10.1137/0213018.

Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959. doi: 10.1147/rd.33.0210.

Jonathan Schaeffer, Robert Lake, Paul Lu, and Martin Bryant. CHINOOK: The world man-machine checkers champion. *AI Magazine*, 17(1):21–29, 1996. doi: 10.1609/aimag.v17i1.1208.

Jonathan Schaeffer, Neil Burch, Yngvi Björnsson, Akihiro Kishimoto, Martin Müller, Robert Lake, Paul Lu, and Steve Sutphen. Checkers is solved. *Science*, 317(5844):1518–1522, 2007. doi: 10.1126/science.1144079.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017. URL `https://arxiv.org/abs/1707.06347`.

Claude E. Shannon. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950. doi: 10.1080/14786445008521796.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815, 2017. URL `https://arxiv.org/abs/1712.01815`.

John Tromp and Gunnar Farnebäck. Combinatorics of Go. Revised version, January 2016. URL `https://tromp.github.io/go/gostate.pdf`.

WCDF. Rules of draughts (checkers). World Checkers Draughts Federation, n.d. URL `https://wcdf.net/rules/rules_of_checkers_english.pdf`.